FINAL REPORT

Frontier AI Observatory

Architecture, engineering decisions and FinOps

Muhammad Ali Aamir (24L-2558)
Muhammad Zohaib (24L-2530)
github.com/zohaib-mzg/Frontier-AI-Observability
October 2026

This report describes the finished pipeline. Four public sources are collected by GitHub Actions, loaded into a Databricks lakehouse through Bronze and Silver, remodelled into a Gold star schema, and presented in a seven-page Power BI dashboard. It documents the architecture, the Gold data model, the reasons behind the engineering choices, how Spark executed the heaviest queries, and what the pipeline costs to run.

AT A GLANCE

SOURCES	Arena leaderboard, CooperBench study, BenchLM, Epoch AI
LAYERS	Landing volume, Bronze (4 tables), Silver (8 tables), Gold (19 tables and 4 audit views)
LARGEST TABLE	`gold.fact_arena_rating`, 2,313,898 rows, 143 MB in 2 files
SCHEDULE	Collector Monday, Wednesday and Friday at 03:00 UTC. Databricks job the same days at 14:00 Pakistan time
TYPICAL RUN	Bronze about 3 min, Silver about 2.6 min, Gold about 3.5 min, in total about 9 minutes on serverless compute
DASHBOARD	7 pages in Power BI, Import mode, 23 Gold tables and views
COST	Free Edition costs nothing. A list-price estimate of an hourly enterprise run is in section 8

SECTION 1

Project and scope

Choosing an AI model means reading evidence that comes from different places, in different units and with different methods. A high score may describe one task only. A low token price says little about an agent that fails often or runs for a long time. The Observatory collects this evidence in one place and keeps each number tied to its source and measurement context.

The question behind the dashboard is: **which AI model should a developer or researcher choose, what does the published evidence say, and can that evidence be trusted?** The project analyses existing public evidence only. It runs no paid inference and no benchmarks.

Data sources

SOURCE	WHAT IT PROVIDES	FORMAT	CADENCE	LICENCE
Arena leaderboard	Rating histories for text, style-controlled text, web development and agent evaluations	Parquet	Irregular	CC BY 4.0
CooperBench	A coding-agent coordination study in four settings: single agent, shared Git, full team, team without protocol	tar.gz of JSON	Historical	Apache 2.0
BenchLM	Models, benchmark definitions and API prices	JSON, 3 exports	1 to 3 times a week	CC BY-NC 4.0
Epoch AI	Model release and training metadata	CSV	Daily	Attribution requested

The schedule follows how often the sources change. Epoch updates daily and BenchLM one to three times a week, so three runs a week catch changes quickly without wasting compute. The first full load landed about 192 MB and held 2.3 million Arena ratings, 17,328 CooperBench files, 2,383 BenchLM records and 3,626 Epoch models.

SECTION 2

Architecture

End-to-end data flow

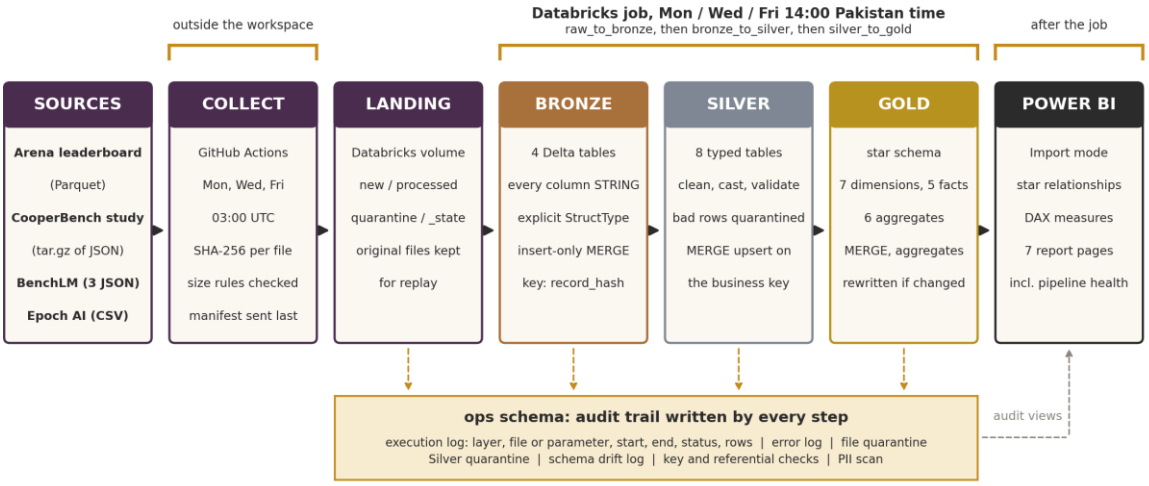


Figure 1. *Data flow from the public sources through Bronze, Silver and Gold to Power BI, with the audit trail of the ops schema.*

Data moves in one direction, and three rules hold at every step.

- **Original files are never changed.** They stay in the landing volume, so Bronze can be rebuilt without downloading again.
- **Every write is a MERGE INTO.** Running a step twice changes nothing.
- **Every step writes an audit row.** Bad data is set aside and logged, never dropped silently.

HOP	WHAT HAPPENS	WHAT IS KEPT AS EVIDENCE
Source to collector	Files are downloaded. A SHA-256 is computed per file. A full batch must be at least 150 MB and an incremental batch at least 1 MB	Collector log, state file
Collector to landing	Files and a manifest are uploaded with the Databricks CLI. The manifest goes last, so a half-finished batch is never loaded	Manifest with checksums
Landing to Bronze	Each file is read against an explicit StructType. Every column is stored as text. Insert-only MERGE on `record_hash`. Loaded files move to `processed/`	Whole source record as JSON, provenance columns
Bronze to Silver	Clean, cast with `try_cast`, validate, de-duplicate, MERGE upsert on the business key	Quarantine log, drift log, cast counts
Silver to Gold	Dimensions and facts by MERGE. Aggregates rewritten only when their content changed. Key and referential checks	Execution log, key check rows, PII scan
Gold to Power BI	Import through a serverless SQL warehouse. Relationships follow the star schema	Audit views inside the dashboard

Layers

LAYER	SCHEMA	CONTENTS	WRITE PATTERN	KEY
Landing	`landing`	A volume with original files, manifests, collector state and logs	Upload	File SHA-256
Bronze	`bronze`	4 tables, every source column as `STRING`, plus provenance and the whole record as JSON	Insert-only MERGE	`record_hash`
Silver	`silver`	8 typed, validated and de-duplicated tables	MERGE upsert	Business key
Gold	`gold`	Star schema: 7 dimensions, 5 facts, 6 aggregates, 1 alias table	MERGE, or overwrite when changed	Surrogate keys
Operations	`ops`	Execution, error, quarantine, drift and Silver quarantine logs	Append	Row id

Orchestration

The collector runs on GitHub Actions every Monday, Wednesday and Friday at 03:00 UTC. A Databricks job then runs on the same days at 14:00 Pakistan time (09:00 UTC) with three tasks in a chain: `raw_to_bronze`, `bronze_to_silver` and `silver_to_gold`. Each task depends on the one before, so Gold starts only after Silver has succeeded, and a failure stops the chain. The job allows one concurrent run, so two loads never write at the same time. When a new incremental file reaches Silver, Gold refreshes in the same run with no manual step.

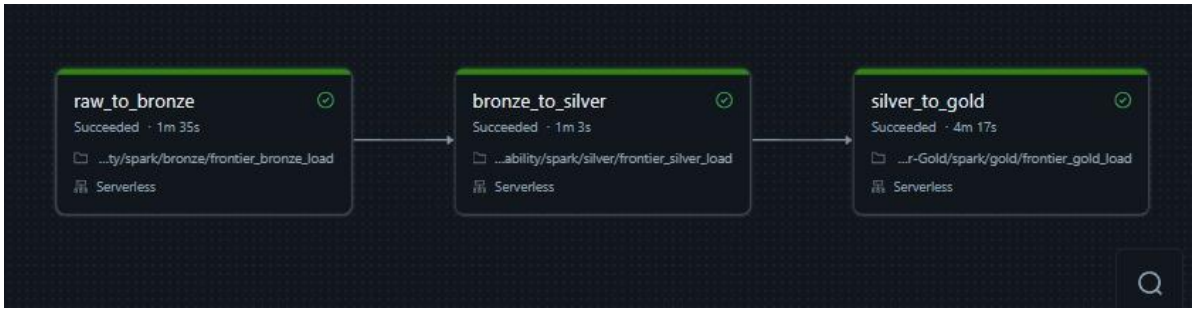


Figure 2. *The job graph. Gold is the third task and depends on Silver.*

Execution logging continues into Gold

The audit framework built for Bronze and Silver now also covers Gold. Every Gold table writes one row to `ops.pipeline_execution_logs` with layer `Silver-to-Gold`, the table as the parameter, start and end time, duration, status and the counts of rows read, inserted, updated and skipped. After the load, primary key and foreign key checks write their own rows, and a PII scan writes a final row. A failing table is logged with its stack trace and does not stop the others.

A	B	C	D	E	F	G	H	I
layer	file_or_param	status	rows_read	rows_inserted	rows_updated	rows_skipped_duplicate	seconds	start_time
Silver-to-Gold (run)	run	SUCCESS	93783	49109	0	43312	222.8	2026-10-24T11:45:01.142Z
Gold data quality (referential integrity)	gold.fact_agent_task_result.evaluated_	SUCCESS	4	0	0	0	1.3	2026-10-24T11:48:41.190Z
Gold data quality (referential integrity)	gold.fact_agent_task_result.agent_mod	SUCCESS	1	0	0	0	1.2	2026-10-24T11:48:38.594Z
Gold data quality (referential integrity)	gold.fact_agent_task_result.task_sk ->	SUCCESS	652	0	0	0	1.3	2026-10-24T11:48:35.957Z
Gold data quality (referential integrity)	gold.fact_agent_task_result.setting_sk	SUCCESS	4	0	0	0	1.3	2026-10-24T11:48:33.407Z
Gold data quality (referential integrity)	gold.fact_capability_score.capability_s	SUCCESS	9	0	0	0	1.3	2026-10-24T11:48:30.734Z
Gold data quality (referential integrity)	gold.fact_capability_score.model_sk ->	SUCCESS	223	0	0	0	1.3	2026-10-24T11:48:28.088Z
Gold data quality (referential integrity)	gold.fact_model_release.publication_d	SUCCESS	1772	0	0	0	1.4	2026-10-24T11:48:25.339Z
Gold data quality (referential integrity)	gold.fact_model_release.model_sk -> d	SUCCESS	3609	0	0	0	1.7	2026-10-24T11:48:22.138Z
Gold data quality (referential integrity)	gold.fact_model_price.valid_from_date	SUCCESS	5	0	0	0	1.2	2026-10-24T11:48:19.193Z
Gold data quality (referential integrity)	gold.fact_model_price.model_sk -> dim	SUCCESS	878	0	0	0	1.4	2026-10-24T11:48:16.552Z
Gold data quality (referential integrity)	gold.fact_arena_rating.date_key -> dim	SUCCESS	305	0	0	0	1.4	2026-10-24T11:48:13.773Z
Gold data quality (referential integrity)	gold.fact_arena_rating.category_sk ->	SUCCESS	64	0	0	0	1.6	2026-10-24T11:48:10.731Z
Gold data quality (referential integrity)	gold.fact_arena_rating.model_sk -> dim	SUCCESS	533	0	0	0	1.7	2026-10-24T11:48:07.594Z
Gold data quality (primary key)	gold.agg_coordination_paired	SUCCESS	652	0	0	0	1.7	2026-10-24T11:48:04.273Z
Gold data quality (primary key)	gold.agg_coordination_by_setting	SUCCESS	4	0	0	0	1.9	2026-10-24T11:48:00.887Z
Gold data quality (primary key)	gold.agg_capability_cost	SUCCESS	357	0	0	0	1.6	2026-10-24T11:47:57.885Z
Gold data quality (primary key)	gold.agg_frontier_timeline	SUCCESS	9134	0	0	0	3	2026-10-24T11:47:53.474Z
Gold data quality (primary key)	gold.agg_style_sensitivity	SUCCESS	11045	0	0	0	1.8	2026-10-24T11:47:49.865Z
Gold data quality (primary key)	gold.agg_arena_latest	SUCCESS	27384	0	0	0	1.5	2026-10-24T11:47:46.682Z
Silver-to-Gold	gold.v_errors	SUCCESS	19	0	0	0	1.6	2026-10-24T11:47:43.750Z
Silver-to-Gold	gold.v_schema_drift	SUCCESS	520	0	0	0	1.8	2026-10-24T11:47:40.498Z
Silver-to-Gold	gold.v_quarantine	SUCCESS	299	0	0	0	2.1	2026-10-24T11:47:36.927Z
Silver-to-Gold	gold.v_pipeline_run	SUCCESS	524	0	0	0	2.2	2026-10-24T11:47:33.331Z
Silver-to-Gold	gold.agg_coordination_paired	SUCCESS	652	652	0	0	3.6	2026-10-24T11:47:28.340Z
Silver-to-Gold	gold.agg_coordination_by_setting	SUCCESS	4	4	0	0	4	2026-10-24T11:47:22.925Z

Figure 3. *Gold log rows: one row per table, one per key check and one for the whole run.*

LAYER VALUE	WRITTEN FOR
`Silver-to-Gold`	Each Gold table, view and aggregate
`Silver-to-Gold (run)`	The whole notebook run, with total rows and duration
`Gold data quality (primary key)`	Each Gold table, counting duplicate and null keys
`Gold data quality (referential integrity)`	Each fact-to-dimension relationship, counting orphan keys
`Gold data quality (PII scan)`	All Gold tables, checking column names and e-mail patterns
`Gold optimize`	The `OPTIMIZE` of the largest fact table

SECTION 3

Gold dimensional model

Gold is not a copy of Silver. It is a star schema built for analytical queries: facts hold the measurable events, and dimensions hold the descriptive attributes by which analysts slice them. Facts and dimensions are joined through integer surrogate keys, and the same dimensions serve several facts.

Facts

FACT TABLE	GRAIN (ONE ROW PER)	MEASURES	BUILT FROM
`fact_arena_rating`	model, category and publication date	rating, confidence bounds, votes, published rank, computed rank and percentile	`arena_leaderboard`
`fact_model_price`	model and price version (type 2)	input, output and cached price per million tokens, context window	`benchlm_pricing`
`fact_model_release`	model release	parameters, training compute, cost, power, hardware, citations	`epoch_models`
`fact_capability_score`	model and capability category	score, verified score	`benchlm_models`
`fact_agent_task_result`	study setting and task	pass flags, test counts, merge status, duration, steps, messages	`cooperbench_*`

Dimensions

DIMENSION	NATURAL KEY	SURROGATE KEY	STRATEGY
`dim_model`	normalised model key	`model_sk`	MERGE, type 1
`dim_date`	calendar date	`date_key` (yyyymmdd)	Generated once, 1950 to 2030
`dim_arena_category`	subset and category	`category_sk`	MERGE
`dim_capability`	BenchLM category	`capability_sk`	Seed
`dim_study_setting`	study setting	`setting_sk`	Seed
`dim_task`	repo, task id, feature pair	`task_sk`	MERGE
`dim_workload_scenario`	scenario name	`scenario_sk`	Seed, editable assumptions

Six aggregate tables (`agg_arena_latest`, `agg_style_sensitivity`, `agg_frontier_timeline`, `agg_capability_cost`, `agg_coordination_by_setting`, `agg_coordination_paired`) hold dashboard-ready summaries that are expensive to compute for each click. A seventh table, `model_alias`, records how each Arena model key was mapped to a base model. Four views over the `ops` tables feed the audit page.

Entity-relationship diagram and keys

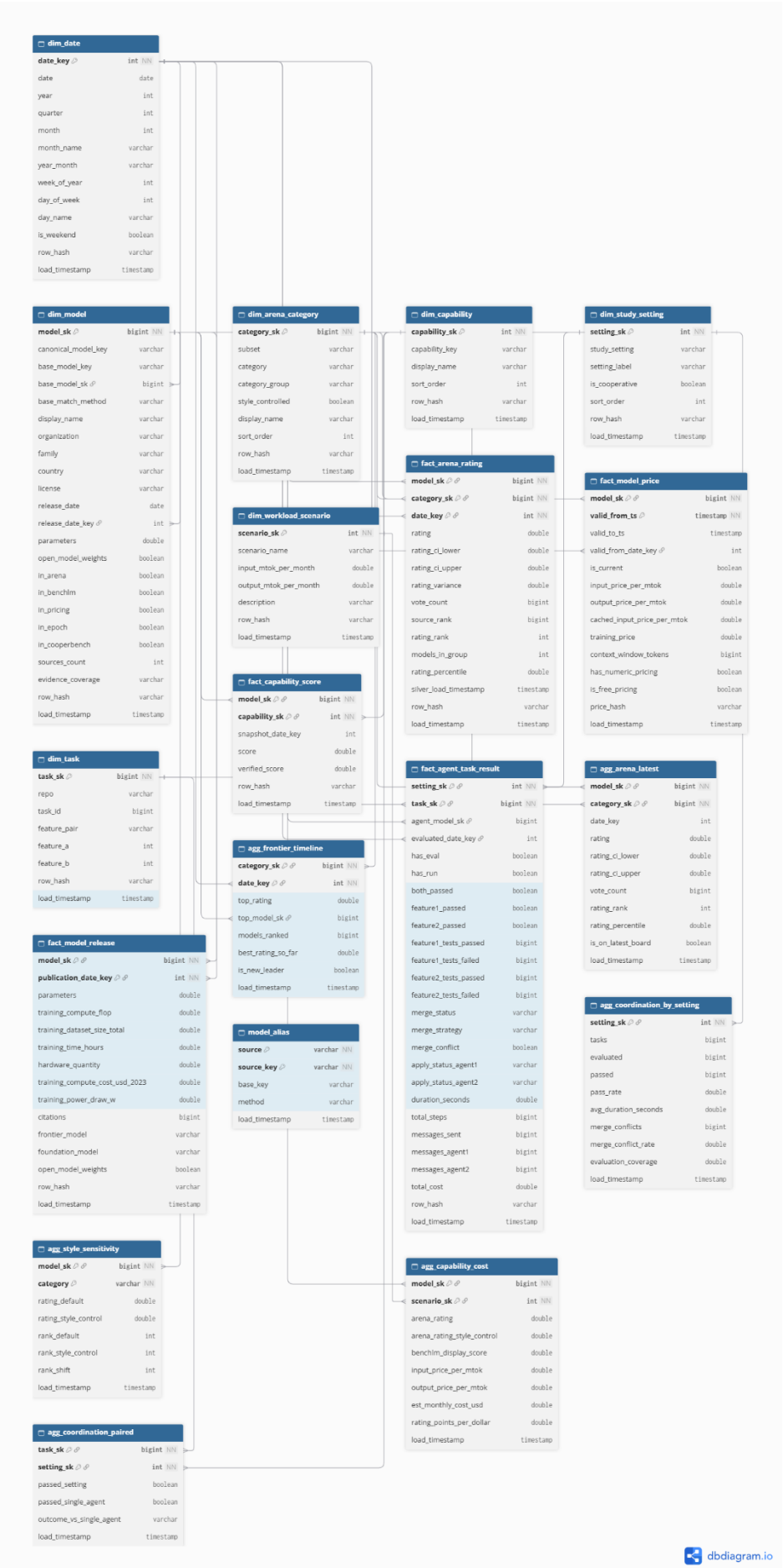

dim_date
date_key int NN
date date
year int
quarter int
month int
month_name varchar
year_month varchar
week_of_year int
day_of_week int
day_name varchar
is_weekend boolean
row_hash varchar
load_timestamp timestamp
dim_model
model_sk bigint NN
canonical_model_key varchar
base_model_key varchar
base_model_sk bigint
base_match_method varchar
display_name varchar
organization varchar
family varchar
country varchar
license varchar
release_date date
release_date_key int
parameters double
open_model_weights boolean
in_arena boolean
in_benchlm boolean
in_pricing boolean
in_epoch boolean
in_cooperbench boolean
sources_count int
evidence_coverage varchar
row_hash varchar
load_timestamp timestamp
dim_arena_category
category_sk bigint NN
subset varchar
category varchar
category_group varchar
style_controlled boolean
display_name varchar
sort_order int
row_hash varchar
load_timestamp timestamp
dim_capability
capability_sk int NN
capability_key varchar
display_name varchar
sort_order int
row_hash varchar
load_timestamp timestamp
dim_study_setting
setting_sk int NN
study_setting varchar
setting_label varchar
is_cooperative boolean
sort_order int
row_hash varchar
load_timestamp timestamp
fact_arena_rating
model_sk bigint NN
category_sk bigint NN
date_key int NN
rating double
rating_ci_lower double
rating_ci_upper double
rating_variance double
vote_count bigint
source_rank bigint
rating_rank int
models_in_group int
rating_percentile double
silver_load_timestamp timestamp
row_hash varchar
load_timestamp timestamp
fact_model_price
model_sk bigint NN
valid_from_ts timestamp NN
valid_to_ts timestamp
valid_from_date_key int
is_current boolean
input_price_per_mtok double
output_price_per_mtok double
cached_input_price_per_mtok double
training_price double
context_window_tokens bigint
has_numeric_pricing boolean
is_free_pricing boolean
price_hash varchar
load_timestamp timestamp
dim_workload_scenario
scenario_sk int NN
scenario_name varchar
input_mtok_per_month double
output_mtok_per_month double
description varchar
row_hash varchar
load_timestamp timestamp
fact_capability_score
model_sk bigint NN
capability_sk int NN
snapshot_date_key int
score double
verified_score double
row_hash varchar
load_timestamp timestamp
dim_task
task_sk bigint NN
repo varchar
task_id bigint
feature_pair varchar
feature_a int
feature_b int
row_hash varchar
load_timestamp timestamp
fact_agent_task_result
setting_sk int NN
task_sk bigint NN
agent_model_sk bigint
evaluated_date_key int
has_eval boolean
has_run boolean
both_passed boolean
feature1_passed boolean
feature2_passed boolean
feature1_tests_passed bigint
feature1_tests_failed bigint
feature2_tests_passed bigint
feature2_tests_failed bigint
merge_status varchar
merge_strategy varchar
merge_conflict boolean
apply_status_agent1 varchar
apply_status_agent2 varchar
duration_seconds double
total_steps bigint
messages_sent bigint
messages_agent1 bigint
messages_agent2 bigint
total_cost double
row_hash varchar
load_timestamp timestamp
agg_arena_latest
model_sk bigint NN
category_sk bigint NN
date_key int
rating double
rating_ci_lower double
rating_ci_upper double
vote_count bigint
rating_rank int
rating_percentile double
is_on_latest_board boolean
load_timestamp timestamp
agg_frontier_timeline
category_sk bigint NN
date_key int NN
top_rating double
top_model_sk bigint
models_ranked bigint
best_rating_so_far double
is_new_leader boolean
load_timestamp timestamp
fact_model_release
model_sk bigint NN
publication_date_key int NN
parameters double
training_compute_flop double
training_dataset_size_total double
training_time_hours double
hardware_quantity double
training_compute_cost_usd_2023 double
training_power_draw_w double
citations bigint
frontier_model varchar
foundation_model varchar
open_model_weights boolean
row_hash varchar
load_timestamp timestamp
model_alias
source varchar NN
source_key varchar NN
base_key varchar
method varchar
load_timestamp timestamp
agg_coordination_by_setting
setting_sk int NN
tasks bigint
evaluated bigint
passed bigint
pass_rate double
avg_duration_seconds double
merge_conflicts bigint
merge_conflict_rate double
evaluation_coverage double
load_timestamp timestamp
agg_style_sensitivity
model_sk bigint NN
category varchar NN
rating_default double
rating_style_control double
rank_default int
rank_style_control int
rank_shift int
load_timestamp timestamp
agg_capability_cost
model_sk bigint NN
scenario_sk int NN
arena_rating double
arena_rating_style_control double
benchlm_display_score double
input_price_per_mtok double
output_price_per_mtok double
est_monthly_cost_usd double
rating_points_per_dollar double
load_timestamp timestamp
agg_coordination_paired
task_sk bigint NN
setting_sk int NN
passed_setting boolean
passed_single_agent boolean
outcome_vs_single_agent varchar
load_timestamp timestamp
dbdiagram.io

Figure 4. *Entity-relationship diagram of the Gold layer. Dimensions in blue, facts in orange, aggregates in green.*

Every relationship is many to one, from a fact or aggregate to a dimension. Surrogate keys are `xxhash64` values of the natural key, so a rerun produces the same keys and no sequence table is needed. Relationships that would create a second filter path in Power BI are kept but marked inactive.

FROM (MANY)	TO (ONE)	IN POWER BI
fact_arena_rating.model_sk	dim_model.model_sk	active
fact_arena_rating.category_sk	dim_arena_category.category_sk	active
fact_arena_rating.date_key	dim_date.date_key	active
fact_model_price.model_sk	dim_model.model_sk	active
fact_model_price.valid_from_date_key	dim_date.date_key	inactive
fact_model_release.model_sk	dim_model.model_sk	active
fact_model_release.publication_date_key	dim_date.date_key	active
fact_capability_score.model_sk	dim_model.model_sk	active
fact_capability_score.capability_sk	dim_capability.capability_sk	active
fact_agent_task_result.setting_sk	dim_study_setting.setting_sk	active
fact_agent_task_result.task_sk	dim_task.task_sk	active
fact_agent_task_result.agent_model_sk	dim_model.model_sk	inactive
fact_agent_task_result.evaluated_date_key	dim_date.date_key	inactive
dim_model.base_model_sk	dim_model.model_sk	self link, not modelled
agg_arena_latest.model_sk	dim_model.model_sk	active
agg_arena_latest.category_sk	dim_arena_category.category_sk	active
agg_style_sensitivity.model_sk	dim_model.model_sk	active
agg_frontier_timeline.category_sk	dim_arena_category.category_sk	active
agg_frontier_timeline.date_key	dim_date.date_key	active
agg_frontier_timeline.top_model_sk	dim_model.model_sk	inactive
agg_capability_cost.model_sk	dim_model.model_sk	active
agg_capability_cost.scenario_sk	dim_workload_scenario.scenario_sk	active
agg_coordination_by_setting.setting_sk	dim_study_setting.setting_sk	active
agg_coordination_paired.task_sk	dim_task.task_sk	active
agg_coordination_paired.setting_sk	dim_study_setting.setting_sk	active

Model identity

Arena publishes model configurations such as `claude-opus-5-5-high`, while BenchLM, the price list and Epoch know the base model `claude-opus-5-5`. `dim_model` therefore keeps one row per configuration and links it to a base model with `base_model_sk`. Ratings attach to the configuration. Prices, benchmark scores and release data attach through the base model.

The matching rule removes only trailing effort words (high, xhigh, max, medium, low, minimal, thinking, reasoning), budget tokens such as 32k, and date stamps, and only when the shorter key exists in another source. It never removes `preview`, because a preview is a different release. Anything else stays unmatched and visible. A reviewed override list in the notebook can accept or refuse a match.

MATCH METHOD	ARENA MODEL KEYS	MEANING
EXACT	141	The key already exists in BenchLM, pricing or Epoch
RULE_STRIP_SUFFIX	116	Matched after removing an effort word, budget or date
RULE_ARENA_GROUP	2	Grouped with another Arena configuration
UNMATCHED	274	Kept under its own key and listed on the coverage page

Load strategy

TABLES	STRATEGY	WHY
Dimensions and facts	`MERGE INTO` on the primary key. A row is updated only when its content hash changed	Idempotent. Unchanged rows are never rewritten
`fact_arena_rating`	Reads only Silver rows whose `load_timestamp` is newer than the last run. Every (category, date) group with a change is recomputed, because ranks need the whole group. The result is staged once, then merged	Gold reads only what changed. The heavy window computation runs once
`fact_model_price`	Type 2 history. A changed price closes the current row and inserts a new version	Price history builds up from the first Gold run
Aggregates and `model_alias`	Overwritten only when a content fingerprint (row count and hash) differs from the stored table	Small tables derived from the facts. A merge would add complexity and no benefit
Audit views	`CREATE OR REPLACE VIEW`	Always current, no storage

Scale

TABLE	ROWS
`fact_arena_rating`	2,313,898
`dim_date`	29,585
`agg_arena_latest`	27,384
`agg_style_sensitivity`	11,045
`agg_frontier_timeline`	9,134
`dim_model`	4,773
`fact_agent_task_result`	2,606
`fact_model_price`	878 (one version per priced model so far)
`dim_task`	652
`agg_capability_cost`	357
`dim_arena_category`	64

SECTION 4

Performance optimization

The largest Gold table, `fact_arena_rating`, uses **liquid clustering on date_key and category_sk**, the two filters that nearly every dashboard query applies: a time window and a category. It is created with `CLUSTER BY (date_key, category_sk)`, and the notebook runs `OPTIMIZE` on it whenever the table changed. Dimension keys are not clustered, because the dimensions are small enough (at most about 4,800 rows) to be read in full.

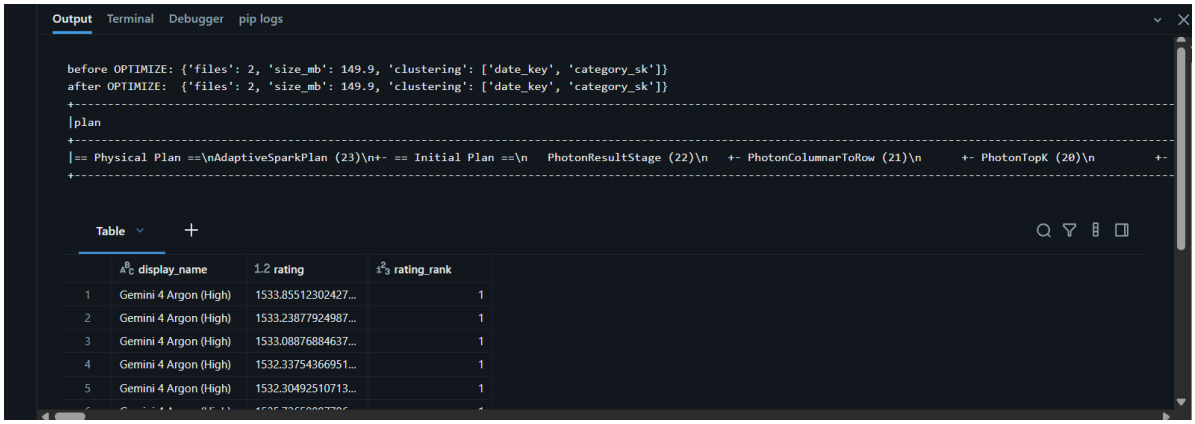


Figure 5. *Layout of `fact_arena_rating` before and after `OPTIMIZE`, and the plan of a dashboard-style query.*

The output shows the table in 2 files of about 149.9 MB with clustering columns `date_key` and `category_sk`, unchanged by `OPTIMIZE`. At this size there is nothing to compact: Delta writes a table of 150 MB as a few large files, so the benefit of clustering (skipping files whose ranges do not match a filter) is limited today. We kept the clustering because the Arena table grows with every board that is published, and a filter on the clustering keys is then pruned by file statistics instead of a full scan. The plan in the same figure shows the query running as `AdaptiveSparkPlan` with Photon operators, and `ORDER BY rating DESC LIMIT 20` executed as a top-K (`PhotonTopK`), which keeps 20 rows in memory instead of sorting 2.3 million.

Other optimizations are in the load logic: Gold reads only changed Arena groups, the heavy fact is staged once instead of being computed twice, unchanged aggregates are not rewritten, and primary key checks run only on tables that changed.

SECTION 5

Business intelligence

Connection and semantic model

Power BI Desktop connects to the `gold` schema with the Databricks connector, using the server hostname and HTTP path of the serverless SQL warehouse (2X-Small) and a personal access token. All 19 Gold tables and the 4 audit views are loaded in **Import mode**, together with one table that holds the DAX measures. The dashboard is therefore as fresh as its last refresh, which is run after the Databricks job.

The Model view reproduces the star schema. Every relationship is one to many with a single filter direction, from the dimension to the fact. dim_date is marked as the date table and also filters the audit views through their date_key. Relationships that would give a table two filter paths (for example the evaluation date of a task, or the price validity date) are kept inactive. Surrogate keys are hidden, and the DAX measures are stored in the _Measures table.

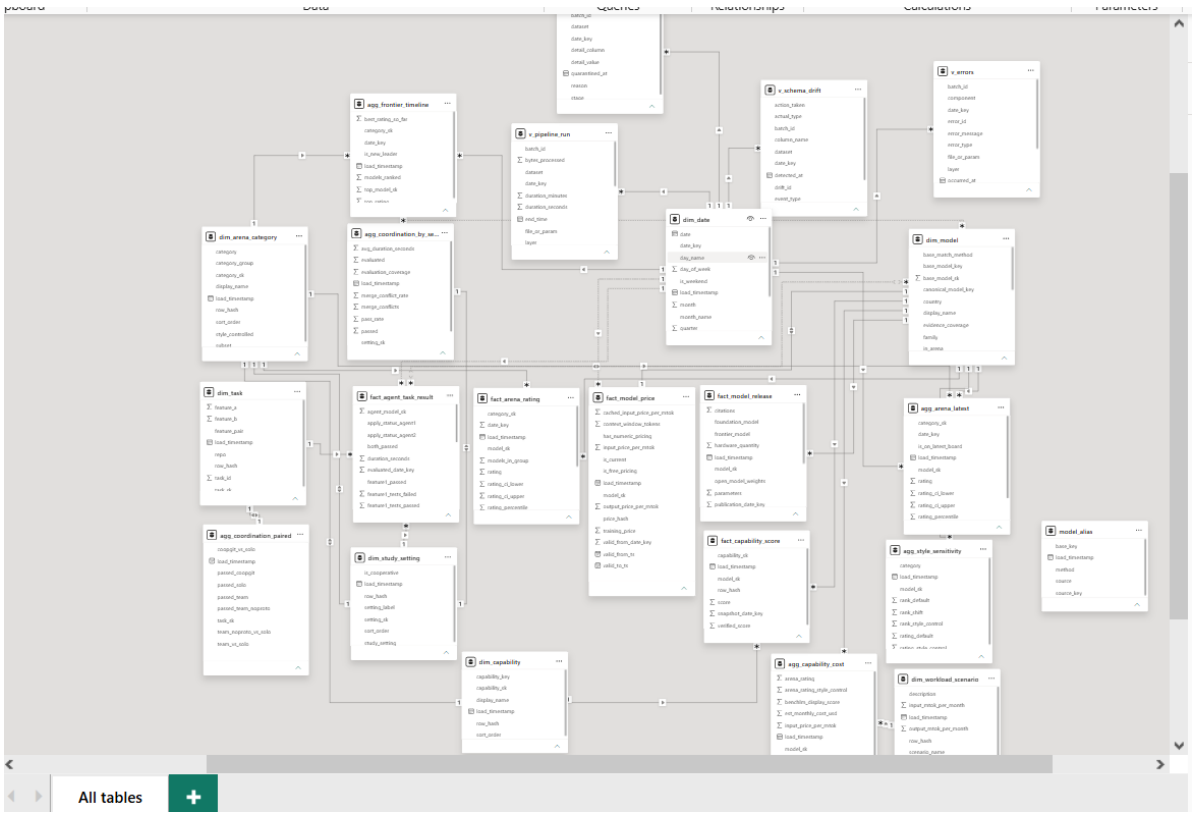


Figure 6. *Power BI Model view of the Gold star schema.*

MEASURE	WHAT IT COMPUTES
`Models Tracked`	Distinct models in dim_model
`Rated Models (text overall)`	Models on the newest text leaderboard
`Overall Rating`	Latest text rating of a model, used to rank models on several pages
`Scenario Cost`	Estimated monthly token cost for the one selected scenario
`Priced Share`	Rated models that have a base-model price, as a share of all rated models
`Task Pass Rate`	Tasks in which both features passed, divided by evaluated tasks
`Merge Conflict Rate`	Tasks with merge conflicts among tasks that have a merge status
`Evaluation Coverage`	Evaluated tasks divided by all tasks

MEASURE	WHAT IT COMPUTES
`Best Rating So Far, Leader Name`	The frontier line and the model that holds it
`Last Successful Run, Failures 30d`	Pipeline health from the execution log

Business questions and where they are answered

BUSINESS QUESTION	PAGE
Which models offer a favourable capability and cost trade-off for a stated workload?	2 Capability and cost
Is a ranking sensitive to the evaluation style or the domain?	3 Rankings and style
When did coordination improve task completion, and what extra time or merge trouble came with it?	4 Coding agent coordination
How has the published frontier changed, and with how much compute?	5 Published capability over time
Which comparisons rest on enough evidence, and which prices are unknown?	6 Evidence coverage
What ran, what was set aside and what changed in the sources?	7 Pipeline health
What does the evidence cover, and who leads today?	1 Overview

The seven pages

PAGE 1 OVERVIEW

Question: what does the evidence cover, and who leads today?

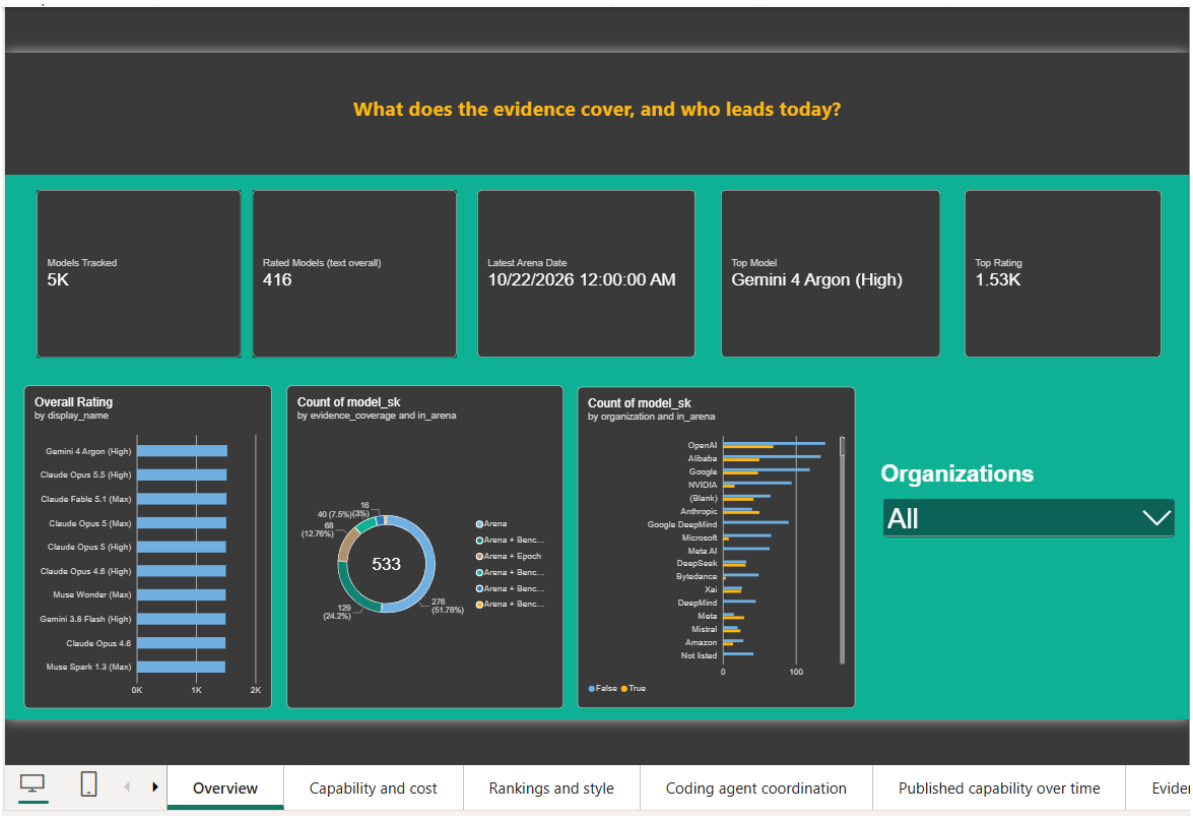


Figure 7. *Page 1. Scale of the evidence and the current leaders.*

The cards give the scale at the time of writing: 4,773 models tracked, 416 of them rated on the latest text leaderboard of 22 October 2026, with Gemini 4 Argon (High) in first place at about 1,533 points. The bar chart ranks the ten highest ratings. The donut shows which other sources know each Arena-rated model, and the organization chart counts rated models per publisher. The organization slicer filters the page.

PAGE 2 CAPABILITY AND COST

Question: which models give the most capability for the money?

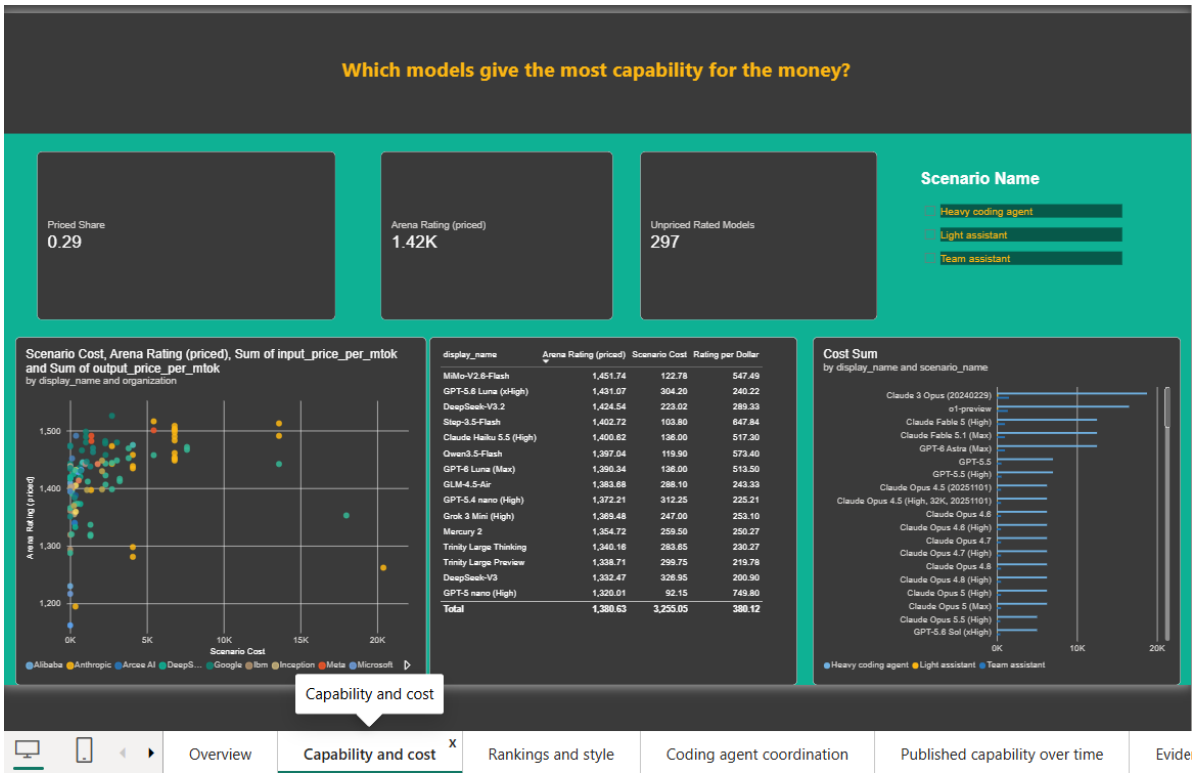


Figure 8. *Page 2. Rating against estimated monthly token cost for the selected workload.*

The scenario slicer chooses one of three monthly token mixes (light assistant, team assistant, heavy coding agent). The scatter plot places each priced model by its estimated monthly cost on a logarithmic axis and its Arena rating. The table lists the best value as rating points per dollar, and the bar chart compares the cost of the same models across the three scenarios. Only 29 percent of the 416 rated models have a base-model price, so 297 are not plotted. They are counted on this page and listed on page 6. A note states that the cost is token spend only, at the list price of the base model, and that a higher reasoning effort can use more tokens.

PAGE 3 RANKINGS AND STYLE

Question: is a model strong everywhere, or only under one style or domain?

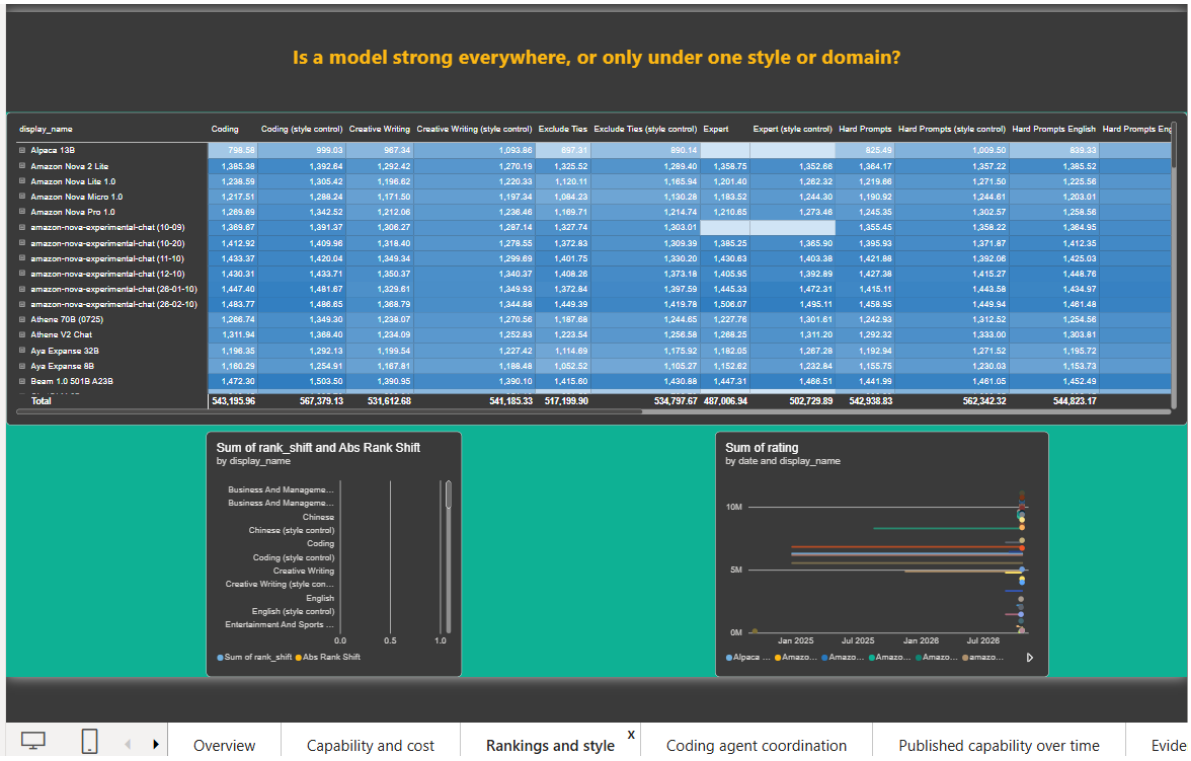


Figure 9. *Page 3. Ratings by domain, rank shift under style control, and trajectories over time.*

The heatmap shows the latest rating of the top models in each capability category, so a model that is strong only in one domain stands out. The bar chart shows the rank shift between the default text board and the style-controlled board: a positive value means the model ranks better when response style is controlled. The line chart follows the rating of the models chosen in the slicer across publication dates.

PAGE 4 CODING AGENT COORDINATION

Question: when did coordination help, and what did it cost?



Figure 10. *Page 4. Pass rate, outcome against the single agent, duration and merge conflicts per study setting.*

All four study settings use the same model on the same 652 tasks, so this page compares coordination methods and not models. The pass rate ranges from about 0.51 for shared Git to about 0.62 for the full team and for the team without protocol verbs, against about 0.56 for a single agent. Average duration differs from about 230 to 280 seconds. The stacked bar compares, task by task, the result of each cooperative setting with the single-agent result. Evaluation coverage is 0.99, and runs without an evaluation count in coverage but not in the pass rate. The cost fields of the study are zero-filled and are not used.

PAGE 5 PUBLISHED CAPABILITY OVER TIME

Question: how has the published frontier moved?



Figure 11. *Page 5. The frontier by date, training compute against release date and releases per quarter.*

The line chart shows the best Arena rating published so far in the chosen category, with markers where a new model took the lead. The scatter plot shows Epoch's training compute estimate against the publication date on a logarithmic axis, split by open and closed weights. The column chart counts releases per quarter, and the bar chart gives the BenchLM capability profile of the models chosen in the slicer.

PAGE 6 EVIDENCE COVERAGE

Question: which comparisons can we trust, and which are missing evidence?

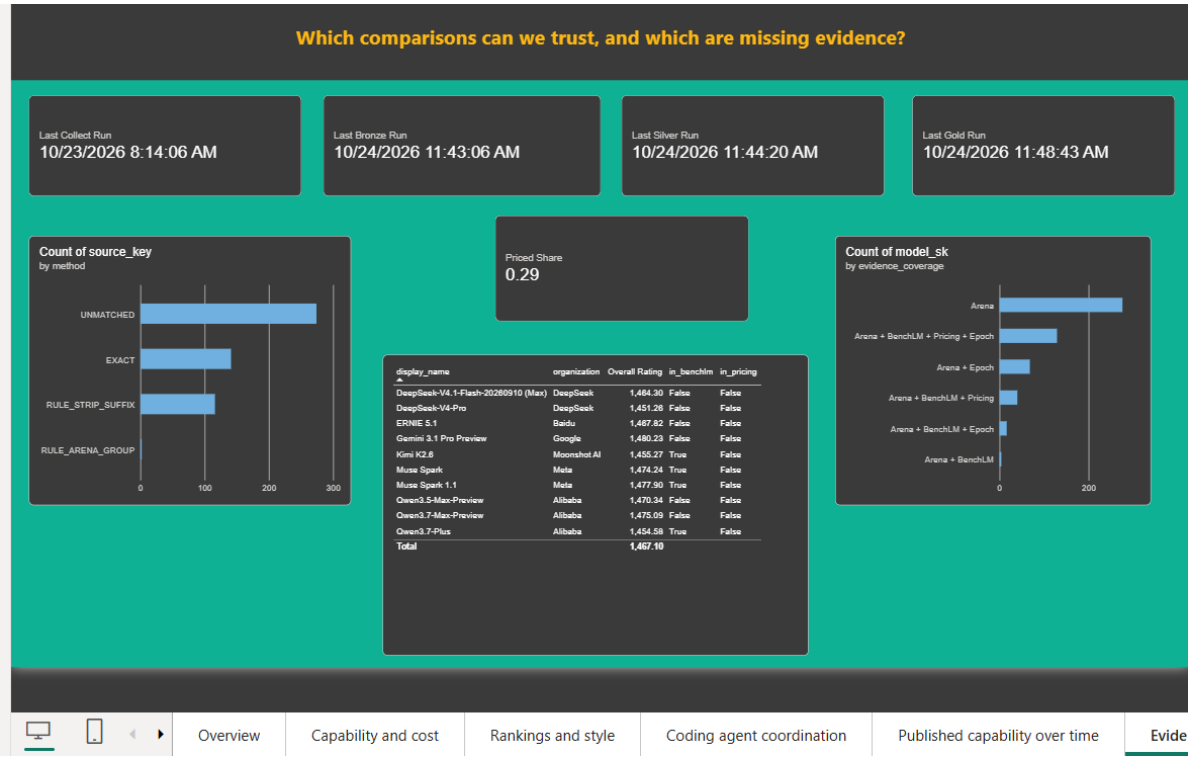


Figure 12. *Page 6. How model names were matched, which sources back each model, and how fresh each layer is.*

This page carries the principle that unknown stays unknown. Of 533 Arena model keys, 141 match exactly, 116 match after a safe rule, 2 are grouped with another Arena configuration and 274 stay unmatched. The stacked bar shows which sources back the Arena-rated models. The table lists the top-rated models that have no price, for example DeepSeek-V4-Pro, ERNIE 5.1 and Gemini 3.1 Pro Preview. The cards show when the collector and each layer last ran successfully.

PAGE 7 PIPELINE HEALTH

Question: can the numbers be trusted?

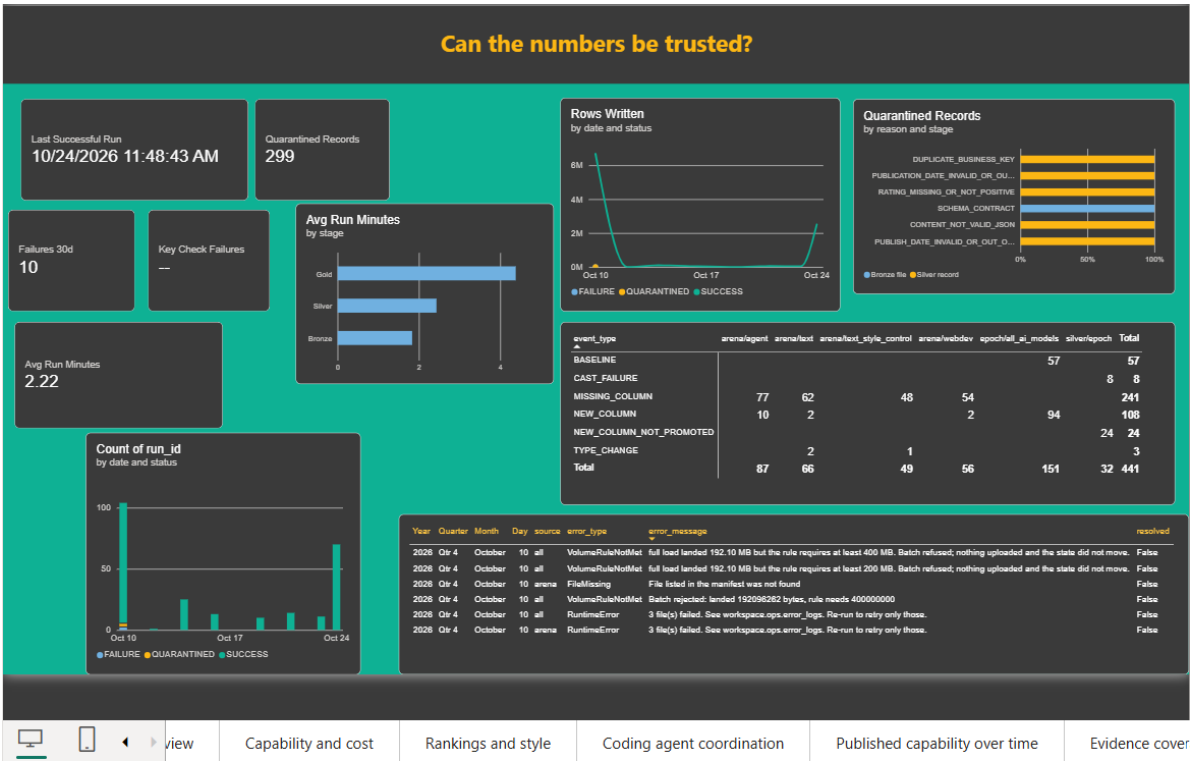


Figure 13. *Page 7. Audit and error logging: runs, rows written, quarantined records, schema changes and errors.*

This page is built on the audit views. At the time of writing the last successful run finished on 24 October 2026 at 11:48, 299 records and files had been set aside across all layers, and the average run took 2.2 minutes. The 10 failures in the last 30 days all date from the first load in October, when the size rules and a missing manifest entry were being corrected (section 6.2). The charts show runs per day by status, rows written per day, average minutes per stage, quarantined records by reason, schema events by type and dataset, and the latest errors.

SECTION 6

Decision log

Each decision records what was chosen, why, and what it costs. The three questions that the project had to answer come first.

6.1 Why these partition and clustering columns?

DECISION	No table is partitioned. fact_arena_rating uses liquid clustering on date_key and category_sk.
WHY NOT PARTITION	Partitioning pays off for tables of terabytes. The largest table here is 143 MB. Partitioning by date would create more than 300 partitions (the fact holds 305 distinct publication dates), each of a few hundred kilobytes. Partitioning by category (64 values, most rows in the text categories) would give skewed partitions. Many small files slow every read.
WHY THESE TWO COLUMNS	Dashboard queries filter by a time window and by a category, and join on the model key. Date and category are the filters that remove the most rows. model_sk is a high-cardinality hash that is better served by the join than by file layout. Liquid clustering lets the keys change later without redesigning the table.
WHAT THE EVIDENCE SHOWS	The table has 2 files and the clustering columns are set (Figure 5). On an unfiltered aggregation, Query Profile itself recommends adding a WHERE clause on the clustering keys (Figure 14). With a date filter the scan produced 248 thousand of 2.31 million rows (Figure 15).
LIMITS	At 150 MB the table cannot show a large pruning gain. A category filter applied through a dimension (for example category_group) cannot prune files by category_sk, only the date part helps. A direct filter on category_sk would prune on both.

6.2 How was schema drift handled when it occurred?

Drift occurred repeatedly in the real data. The pipeline did not fail on any of it. Every event is a row in ops.schema_drift_log, and the numbers below come from that table.

EVENT	COUNT	WHERE IT OCCURRED	HANDLING
BASELINE	57	Epoch CSV	The 57 columns were recorded as the starting schema
NEW_COLUMN	108	Arena agent 10, text 2, web development 2, Epoch 94	ALTER TABLE ADD COLUMNS as STRING, the load continued
MISSING_COLUMN	241	The four Arena subsets	The column was filled with null
TYPE_CHANGE	3	Arena text 2, text with style control 1	Stored as text, so nothing could fail
CAST_FAILURE	8	Silver, Epoch dataset size	Values holding two numbers (for example 929000,929000) became null, with the count logged

EVENT	COUNT	WHERE IT OCCURRED	HANDLING
NEW_COLUMN_NOT_PROMOTED	24	Silver, Epoch	Not carried to Silver until one line is added to the column map

- **Bronze evolves, Silver decides.** Bronze stores text, so a new column or a changed type is absorbed by an explicit ALTER TABLE and logged. Meaning is decided in Silver, where one bad value affects one record and not the batch.
- **`ALTER TABLE ADD COLUMNS` instead of `mergeSchema`.** Every schema change is an explicit, logged statement and cannot happen as a side effect of a write.
- **Gold is insulated.** Gold reads only the Silver columns it names. A new Silver column stays invisible to Gold until a contract line is added and the table is altered, which is a reviewed code change.
- **Arena subsets arrived in stages.** The web development and agent subsets added seven columns to the Arena table (for example rating_lower and score). Silver maps all of them onto one rating and one pair of confidence columns.

Operational failures were also logged and used to correct rules. The error log holds, among others, a full load of 192.10 MB that was refused by a size rule of 400 MB (the rule is now 150 MB), a file listed in a manifest that was not found, and a run in which three files failed and only those were retried. The Arena rating rule rejected negative scores until it was found that the agent subset publishes negative scores by design. The rule now applies outside that subset, and 666 records moved from quarantine into Silver.

6.3 How was personal data managed?

LAYER	HANDLING
Collector	Only public files are downloaded. The Databricks token lives in GitHub Actions secrets and never in the repository
Landing and Bronze	Original files and complete records stay inside the Databricks workspace, which only the two project members can open. CooperBench archives can contain names, e-mail addresses and file paths in traces and patches. That content stays in Bronze raw_content and goes no further
Silver	CooperBench tables hold identifiers, paths, outcomes, durations and costs, with no trace text, test output or patches. The Epoch and BenchLM tables keep an authors column. These are public bibliographic names, kept for traceability inside the restricted workspace
Gold	Gold selects its columns by name. It carries no author names, references, notes, links or free text. Organization columns hold company names, not people
Power BI	The file contains only Gold tables and audit views. Credentials are not stored in the file

Dropping a column was preferred to hashing it, because personal identifiers add nothing to the planned metrics, and a hash of a name is still personal data. The PII scan cell of the Gold notebook checks the result: it lists any Gold column whose name suggests personal data and counts e-mail patterns in every text column of every Gold table, then writes the outcome to the execution log.

RESULT OF THE PII SCAN

[Insert the printed result of Cell 10 here, for example: Gold columns with a suspicious name: none. Gold text columns with an e-mail pattern: none.]

6.4 Other decisions

DECISION	REASON	ALTERNATIVE REJECTED
Bronze stores text only	A type change in a source can never reject a file	A typed Bronze table with schema inference
Explicit StructType, no inferSchema	A reviewed contract, no extra pass over the data, no wrong guesses	Schema inference
MERGE INTO for every write	Reruns and backfills cannot create duplicates	Plain append
Content hash decides updates	A row is rewritten only if it changed, so load_timestamp stays meaningful	Update on every run
Gold reads Silver rows by load_timestamp	Silver changes it only on insert or update, so it is a reliable change marker	Recompute the whole fact every run
One dimension row per model configuration, linked to a base model	A rating belongs to a configuration, a price to the base model, and reasoning settings must stay distinguishable	Merge all configurations into one model
Surrogate keys as xxhash64	Deterministic, no sequence table, fast integer joins in Power BI	Text keys or identity columns
Type 1 dimensions, type 2 price history	Attributes need only the latest value, price changes are worth keeping	Type 2 everywhere
Aggregates replaced only when changed	They are small and derived. Unchanged tables are not rewritten	MERGE on aggregates
Import mode in Power BI	The 2X-Small warehouse stops when idle and Import keeps every click fast	DirectQuery
Unknown values stay null	No invented zeros. Unpriced models are shown on the coverage page	Zero fill
Deletions are not applied	A record that disappears from a snapshot is not proof that it never existed	Delete from Silver
Query Profile instead of Spark UI	Free Edition runs on serverless compute, which has no Spark UI	A classic cluster, not available

SECTION 7

Spark execution and DAG analysis

Free Edition offers serverless compute only, and serverless compute has no Spark UI. The Query Profile of Databricks shows the same execution plan as a graph of operators with rows, time and memory per operator, so the analysis below uses it. Both queries run on the serverless SQL warehouse, which executes with the Photon engine.

7.1 Aggregation and window: the frontier timeline

The first query has the shape of the most complex Gold transformation: the best rating per category and date, followed by a running maximum per category (agg_frontier_timeline). It reads the 2.3 million rows of the largest fact table.

```
SELECT category_sk, date_key, top_rating,
       MAX(top_rating) OVER (PARTITION BY category_sk ORDER BY date_key
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
best_rating_so_far
FROM (SELECT category_sk, date_key, MAX(rating) AS top_rating
      FROM workspace.gold.fact_arena_rating GROUP BY category_sk, date_key)
```

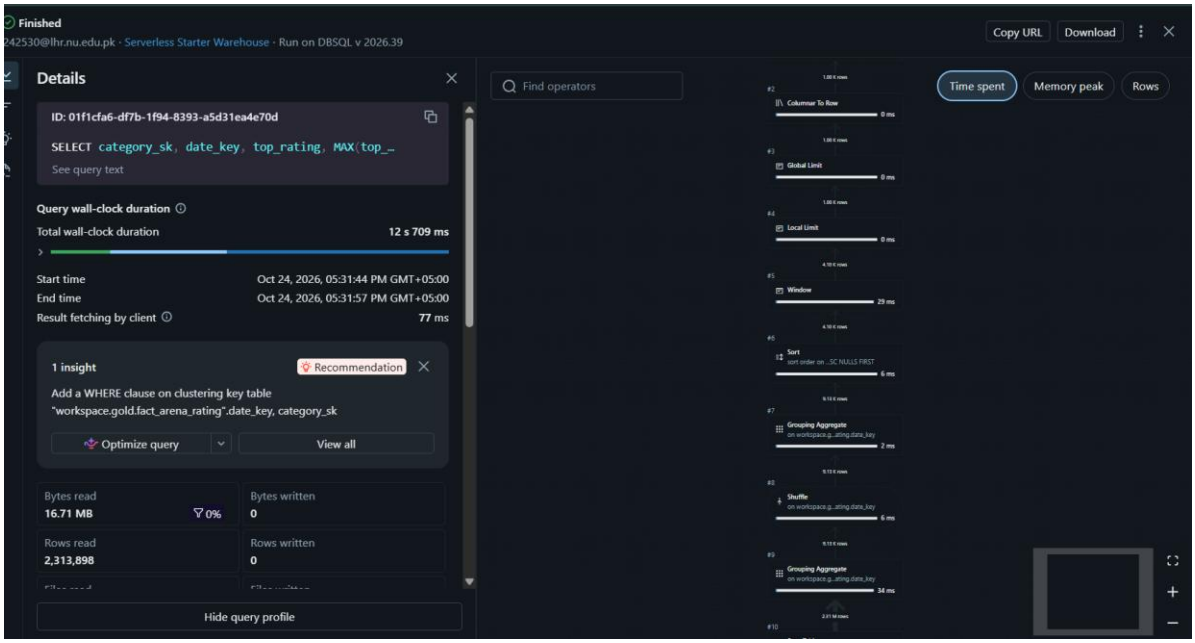


Figure 14. *Query Profile of the frontier aggregation, read from the bottom to the top.*

OPERATOR	WHAT SPARK DID	OBSERVED
Scan	Read only the three columns the query needs	16.71 MB read of a 149.9 MB table, 2,313,898 rows
Grouping Aggregate (partial)	Grouped inside each partition before moving any data	2.31 million rows in, 9.13 thousand rows out
Shuffle	Redistributed the partial results by the grouping keys so equal keys meet	Only 9.13 thousand rows crossed the network
Grouping Aggregate (final)	Combined the partial groups into one row per category and date	9.13 thousand rows
Sort and Window	Ordered by date within each category and computed the running maximum	Window 29 ms

OPERATOR	WHAT SPARK DID	OBSERVED
Limit, Columnar To Row	Applied the 1,000-row limit of the SQL editor and converted columns to rows for the client	Under 1 ms each

Three ideas explain the plan. **Column pruning** limits the scan to 11 percent of the table's bytes. **Partial aggregation** shrinks 2.31 million rows to 9.13 thousand before the **shuffle**, which is the expensive step in a distributed query because it moves data between machines, so the shuffle is almost free here. The **window** needs all rows of a category together and ordered by date, which is why the sort and the exchange precede it. The operators in the visible part each ran for tens of milliseconds, so most of the 12.7 seconds of wall-clock time is spent outside them, in scheduling and in reading the files.

The profile also carries a recommendation: add a WHERE clause on the clustering keys. The query reads the whole table by design, because it builds a timeline of every date. The queries behind the dashboard visuals filter by date and category, where the clustering applies.

7.2 Star join with a filter: ratings by capability category

```
SELECT c.display_name AS category, m.organization, COUNT(*) AS ratings, ROUND(AVG(f.rating),
1) AS avg_rating
FROM workspace.gold.fact_arena_rating f
JOIN workspace.gold.dim_model m ON f.model_sk = m.model_sk
JOIN workspace.gold.dim_arena_category c ON f.category_sk = c.category_sk
WHERE f.date_key >= 20260901 AND c.category_group = 'Capability'
GROUP BY c.display_name, m.organization
```

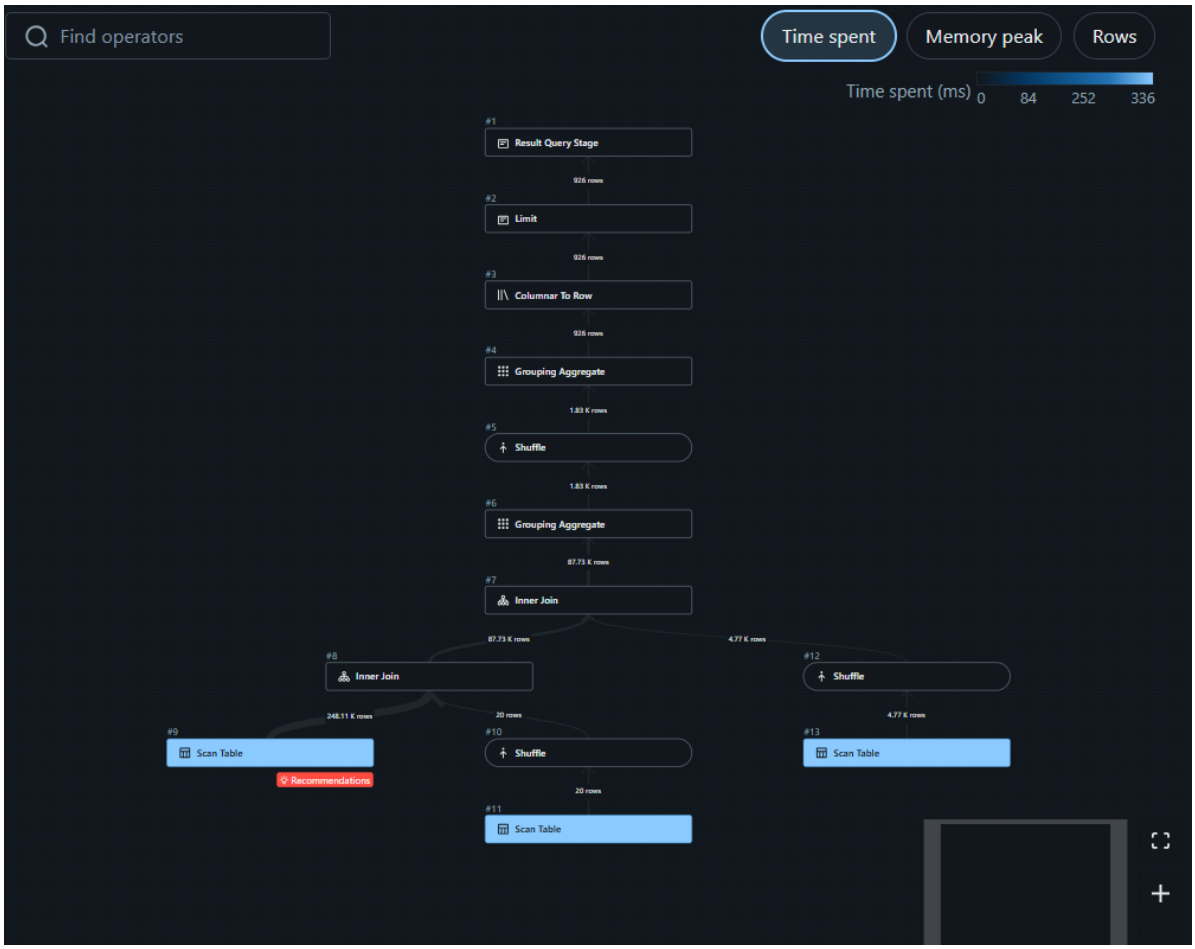


Figure 15. *Query Profile of a star join: a fact table and two dimensions, with a date filter and a category-group filter.*

OPERATOR	WHAT SPARK DID	OBSERVED
Scan of `fact_arena_rating`	Applied the date filter while scanning	248.11 thousand rows left of 2.31 million
Scan of `dim_arena_category`	Applied the category-group filter on the small dimension	20 of 64 categories
Inner Join (fact and category)	Joined the filtered fact rows with the 20 categories	87.73 thousand rows
Scan of `dim_model` **and Inner Join**	Joined with the model dimension to get the organization	4.77 thousand dimension rows, 87.73 thousand rows out
Grouping Aggregate (partial), Shuffle, Grouping Aggregate (final)	Counted and averaged per category and organization	87.73 thousand rows reduced to 1.83 thousand before the shuffle, then to 926 rows

This plan shows **predicate pushdown**: the date condition is evaluated inside the scan, so about nine out of ten rows of the fact table never leave it. The condition on category_group is applied to the 64-row dimension first, and the join then keeps only the fact rows of those 20 categories, which is why the star schema keeps the big table narrow in the join. The small dimension inputs (20 and 4,770 rows) are cheap to exchange. Partial aggregation again reduces the data before the final shuffle. The scan of the fact table carries a recommendation flag for the same reason as in 7.1: the category filter reaches the fact only through the join, so the file layout is used for the date but not for the category.

7.3 The dashboard probe

The top-20 query of Figure 5 runs as an adaptive plan with Photon operators. ORDER BY rating DESC LIMIT 20 is executed as a top-K, which keeps 20 rows in memory instead of sorting 2.3 million rows. Adaptive query execution lets Spark change the plan at run time, for example the number of shuffle partitions, from the sizes it actually observes.

CONCEPT	WHERE IT APPEARED	EFFECT
Column pruning	Query A scan: 3 of 13 columns	Read 11 percent of the bytes
Predicate pushdown	Query B scan: date filter	2.31 million rows reduced to 248 thousand at the scan
Partial aggregation	Both queries	Shuffle moves thousands of rows instead of millions
Shuffle	Both queries	The data exchange between stages, the main cost of wide operations
Window function	Query A	Needs sorted groups, so sort and exchange come first
Star join	Query B	Filter the small dimension, then join the big fact
Top-K	Figure 5 plan	Keeps only the best 20 rows
Adaptive execution, Photon	Figure 5 plan	Run-time plan changes and a vectorised engine

SECTION 8

Cost and FinOps

8.1 What is billed and what was measured

The project runs on Databricks Free Edition and GitHub Actions, so no money was spent and no credits were consumed. The workspace shows no usage or billing page, and the cost figures below are therefore **estimates**. They combine run times measured from the execution log with stated assumptions about the billing rate. A DBU is the unit in which Databricks bills compute. Serverless jobs are billed per DBU consumed, and the DBU consumption per hour of a run is not visible on Free Edition. The estimates bracket it with three assumed averages.

8.2 Measured run times

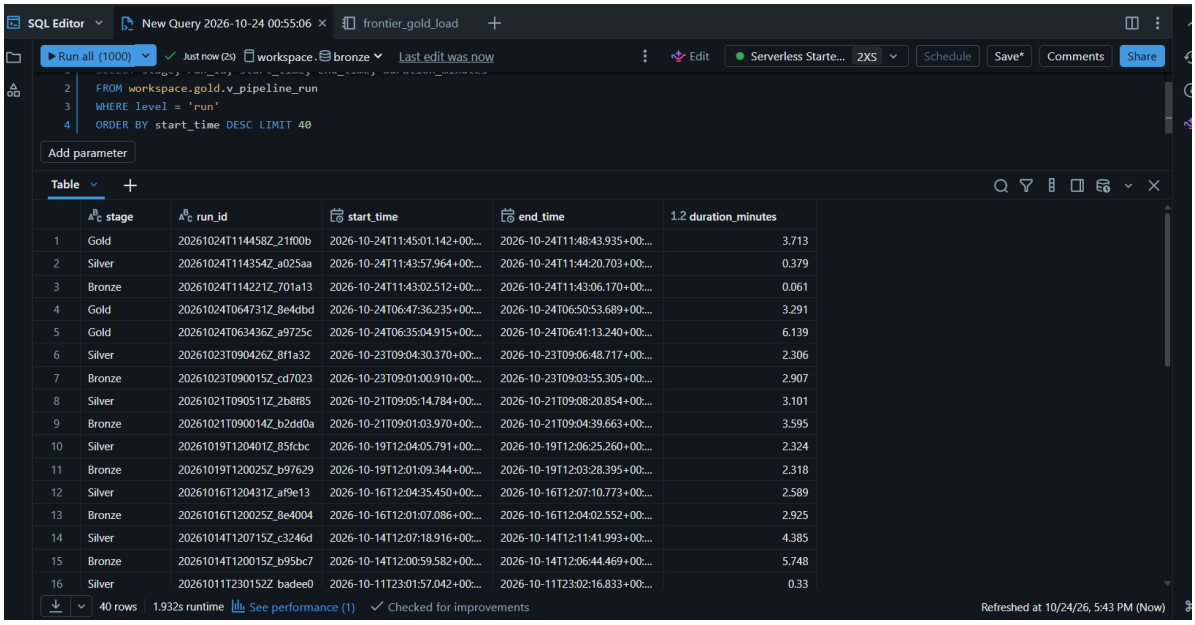


Figure 16. *Run durations by stage from the execution log.*

STAGE	RUNS WITH NEW DATA (MINUTES)	TYPICAL	RUN WITH NOTHING NEW
Bronze	2.3, 2.9, 2.9, 3.6, 5.7	about 2.9	0.06
Silver	2.3, 2.3, 2.6, 3.1, 4.4	about 2.6	0.38
Gold	3.3, 3.7 (first build 6.1)	about 3.5	3.3 to 3.7

A typical scheduled run with new data therefore takes about **9 minutes** (2.9 + 2.6 + 3.5), or 0.15 hours. A run with nothing new takes under 4 minutes, and almost all of it is Gold, which still rewrites aggregates and runs its key checks (this is one of the optimizations in 8.5).

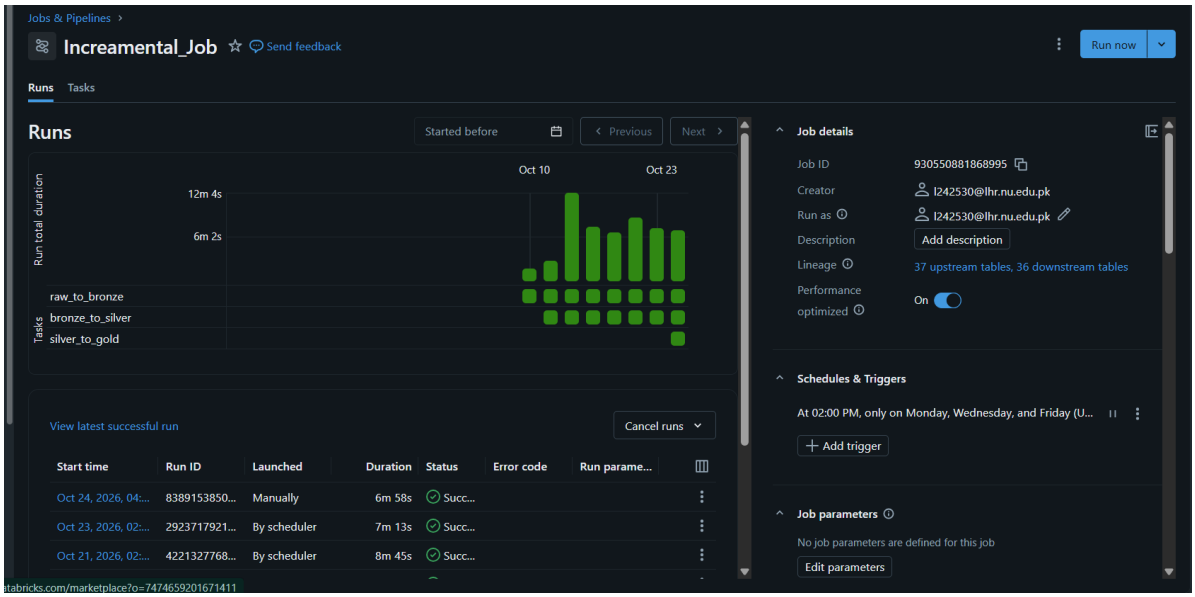


Figure 17. *Job run history and the duration of each task.*

The job page shows the same picture from the scheduler. A run without new data took 6 minutes 58 seconds in total, of which the Gold task took 4 minutes 17 seconds. Runs before Gold was added took 7 to 9 minutes, including serverless start-up time.

8.3 Storage

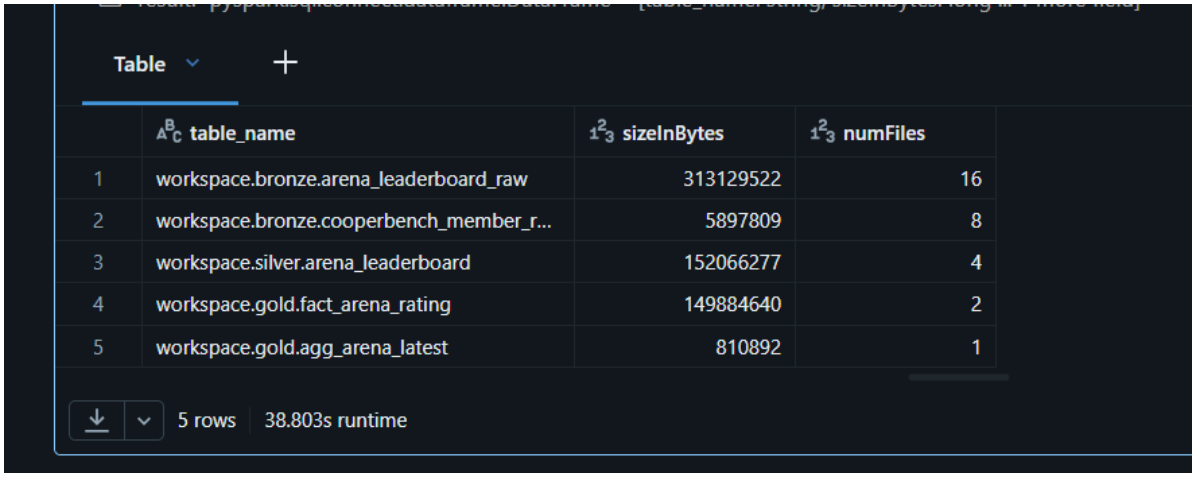


Figure 18. *Size and number of files of the largest tables.*

TABLE	SIZE	FILES
bronze.arena_leaderboard_raw	313.1 MB	16
bronze.cooperbench_member_raw	5.9 MB	8
silver.arena_leaderboard	152.1 MB	4
gold.fact_arena_rating	149.9 MB	2
gold.agg_arena_latest	0.8 MB	1
Total of these five	621.8 MB	

The whole lakehouse holds about 0.7 GB. At a typical object-storage price of about $0.023 per GB and month, storage costs less than two cents a month, so compute dominates every estimate.

8.4 Estimate: the pipeline run hourly by an enterprise for a year

The estimate multiplies run time by a DBU rate and a price per DBU. The price used is **$0.35 per DBU** for serverless jobs (a published list price for the premium tier in a US region), which must be checked against the price list of the chosen cloud and region. A year has 8,760 hourly runs.

ASSUMED AVERAGE	COST OF ONE 9-MINUTE RUN	THREE RUNS A WEEK (156 A YEAR)	DAILY (365)	HOURLY (8,760)
4 DBU per hour (low)	$0.21	$33	$77	$1,840
8 DBU per hour (middle)	$0.42	$66	$153	$3,679
16 DBU per hour (high)	$0.84	$131	$307	$7,358

ITEM	ASSUMPTION	ANNUAL COST, HOURLY
Pipeline compute, middle case	9 minutes, 8 DBU per hour, $0.35 per DBU	$3,679
Larger enterprise data	10 times the data and 3 times the run time (27 minutes)	about $11,000
Dashboard refresh	Serverless SQL, 2X-Small = 4 DBU per hour at $0.70 per DBU, 5 minutes per refresh	$2,044 (daily refresh: $85)
Storage	About 0.7 GB at $0.023 per GB and month	under $1
Power BI licences	Not included	not estimated

READING THE ESTIMATE

The run time is mostly fixed overhead (start-up, checks, logging) and not data volume. Hourly processing is therefore expensive for sources that change three times a week. The current design already costs about $66 a year in the middle case, and the same design run hourly costs about $3,700, although the data would not be any fresher.

8.5 How to reduce the cost

LEVER	EFFECT	STATUS
Run when data arrives, not on a clock	The number of runs follows the sources: 156 a year instead of 8,760 at the current cadence	Proposed
Exit early when Silver did not change	A run with nothing new drops from about 3.7 to about 0.9 minutes. Hourly cost in the middle case falls from about $1,530 to about $380 a year	Proposed
Skip unchanged aggregate writes and key checks on unchanged tables	Fewer rewrites and scans	Done
Process only changed Arena groups and stage the fact once	The 2.3 million rows are not recomputed twice	Done
Standard (not performance-optimized) serverless mode for non-urgent jobs	A lower DBU rate for a slower start. The job page shows performance optimization on	Proposed
Run OPTIMIZE and VACUUM weekly, not on every run	Compaction cost follows table growth, not the clock	Partly

LEVER	EFFECT	STATUS
Refresh Power BI only after the job succeeded	Warehouse time is spent only when data changed: about $36 a year at three refreshes a week	Proposed
Committed-use discount	A contract rate for steady enterprise use	Enterprise option

SECTION 9

Quality evidence and limits

EVIDENCE	RESULT
Gold rerun on unchanged data	Dimensions and facts: 0 inserted and 0 updated. The Arena fact read 0 rows, because no Silver row had changed since the last run. Unchanged aggregates are not rewritten
Primary key checks	0 duplicate keys and 0 null keys in every Gold table
Referential integrity checks	0 orphan keys in all 13 fact-to-dimension relationships
Silver idempotency	A forced rerun of all eight tables inserted and updated 0 rows (Silver proof notebooks 03 to 05)
Backfill	A run with from_batch and to_batch processed only the selected batches (Silver proof 06)
Set-aside records	299 records and files across all layers, each with its reason and original value in the quarantine logs
Personal data	Gold selects its columns by name. The PII scan result is in section 6.3

Limits

- **Deletions are not applied.** A record that disappears from a snapshot stays in Silver. The Bronze snapshots keep the evidence.
- **Name matching is partial.** Of 533 Arena model keys, 274 stay unmatched on purpose, and only 29 percent of rated models have a price. Cost comparisons cover the priced models only.
- **Cost is token spend only.** It uses the list price of the base model and a stated monthly token mix. It is not the bill of an agent run.
- **One model in the agent study.** The coordination page compares settings, not models.
- **Clustering shows little at this size.** The benefit grows with the table. Query Profile replaces the Spark UI, which serverless compute does not offer.
- **FinOps figures are estimates.** Free Edition shows no bill. The DBU rate and price per DBU are assumptions that must be replaced by billing data in a paid workspace.

APPENDIX A

Gold data dictionary

Generated from the table contracts in the notebook. Primary keys (PK) and foreign keys (FK) are marked. `technical` columns support the merge and the audit.

`gold.dim_date` dimension

One calendar day from 1950 to 2030.

COLUMN	TYPE	ROLE
date_key	int	PK
date	date	attribute
year	int	attribute
quarter	int	attribute
month	int	attribute
month_name	string	attribute
year_month	string	attribute
week_of_year	int	attribute
day_of_week	int	attribute
day_name	string	attribute
is_weekend	boolean	flag
row_hash	string	technical
load_timestamp	timestamp	technical

`gold.dim_model` dimension

One model configuration. base_model_sk points to the model that BenchLM, pricing and Epoch know.

COLUMN	TYPE	ROLE
model_sk	bigint	PK
canonical_model_key	string	attribute
base_model_key	string	attribute
base_model_sk	bigint	FK to dim_model.model_sk
base_match_method	string	attribute
display_name	string	attribute
organization	string	attribute
family	string	attribute
country	string	attribute
license	string	attribute
release_date	date	attribute
release_date_key	int	attribute
parameters	double	attribute
open_model_weights	boolean	flag
in_arena	boolean	flag
in_benchlm	boolean	flag
in_pricing	boolean	flag

COLUMN	TYPE	ROLE
in_epoch	boolean	flag
in_cooperbench	boolean	flag
sources_count	int	attribute
evidence_coverage	string	attribute
row_hash	string	technical
load_timestamp	timestamp	technical

gold.dim_arena_category dimension

One Arena subset and category.

COLUMN	TYPE	ROLE
category_sk	bigint	PK
subset	string	attribute
category	string	attribute
category_group	string	attribute
style_controlled	boolean	flag
display_name	string	attribute
sort_order	int	attribute
row_hash	string	technical
load_timestamp	timestamp	technical

gold.dim_capability dimension

One BenchLM capability category, plus the overall score.

COLUMN	TYPE	ROLE
capability_sk	int	PK
capability_key	string	attribute
display_name	string	attribute
sort_order	int	attribute
row_hash	string	technical
load_timestamp	timestamp	technical

gold.dim_study_setting dimension

One CooperBench study setting.

COLUMN	TYPE	ROLE
setting_sk	int	PK
study_setting	string	attribute
setting_label	string	attribute
is_cooperative	boolean	flag
sort_order	int	attribute
row_hash	string	technical
load_timestamp	timestamp	technical

gold.dim_task dimension

One CooperBench task and feature pair.

COLUMN	TYPE	ROLE
task_sk	bigint	PK
repo	string	attribute
task_id	bigint	attribute
feature_pair	string	attribute
feature_a	int	attribute
feature_b	int	attribute
row_hash	string	technical
load_timestamp	timestamp	technical

gold.dim_workload_scenario dimension

One monthly token-mix assumption used for cost estimates.

COLUMN	TYPE	ROLE
scenario_sk	int	PK
scenario_name	string	attribute
input_mtok_per_month	double	attribute
output_mtok_per_month	double	attribute
description	string	attribute
row_hash	string	technical
load_timestamp	timestamp	technical

gold.fact_arena_rating fact

One rating of one model in one category on one publication date. Clustered by date_key, category_sk.

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
category_sk	bigint	PK, FK to dim_arena_category.category_sk
date_key	int	PK, FK to dim_date.date_key
rating	double	measure
rating_ci_lower	double	measure
rating_ci_upper	double	measure
rating_variance	double	measure
vote_count	bigint	measure
source_rank	bigint	measure
rating_rank	int	measure
models_in_group	int	measure
rating_percentile	double	measure
silver_load_timestamp	timestamp	lineage
row_hash	string	technical
load_timestamp	timestamp	technical

gold.fact_model_price fact, type 2

One price version of one model (slowly changing, type 2).

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
valid_from_ts	timestamp	PK
valid_to_ts	timestamp	attribute
valid_from_date_key	int	FK to dim_date.date_key
is_current	boolean	flag
input_price_per_mtok	double	measure
output_price_per_mtok	double	measure
cached_input_price_per_mtok	double	measure
training_price	double	measure
context_window_tokens	bigint	measure
has_numeric_pricing	boolean	flag
is_free_pricing	boolean	flag
price_hash	string	attribute
load_timestamp	timestamp	technical

gold.fact_model_release fact

One model release published by Epoch AI.

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
publication_date_key	int	PK, FK to dim_date.date_key
parameters	double	measure
training_compute_flop	double	measure
training_dataset_size_total	double	measure
training_time_hours	double	measure
hardware_quantity	double	measure
training_compute_cost_usd_2023	double	measure
training_power_draw_w	double	measure
citations	bigint	measure
frontier_model	string	attribute
foundation_model	string	attribute
open_model_weights	boolean	flag
row_hash	string	technical
load_timestamp	timestamp	technical

gold.fact_capability_score fact

One BenchLM score of one model in one capability category.

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
capability_sk	int	PK, FK to dim_capability.capability_sk

COLUMN	TYPE	ROLE
snapshot_date_key	int	measure
score	double	measure
verified_score	double	measure
row_hash	string	technical
load_timestamp	timestamp	technical

gold.fact_agent_task_result fact

One task in one study setting. Runs without an evaluation stay visible.

COLUMN	TYPE	ROLE
setting_sk	int	PK, FK to dim_study_setting.setting_sk
task_sk	bigint	PK, FK to dim_task.task_sk
agent_model_sk	bigint	FK to dim_model.model_sk
evaluated_date_key	int	FK to dim_date.date_key
has_eval	boolean	flag
has_run	boolean	flag
both_passed	boolean	flag
feature1_passed	boolean	flag
feature2_passed	boolean	flag
feature1_tests_passed	bigint	measure
feature1_tests_failed	bigint	measure
feature2_tests_passed	bigint	measure
feature2_tests_failed	bigint	measure
merge_status	string	attribute
merge_strategy	string	attribute
merge_conflict	boolean	flag
apply_status_agent1	string	attribute
apply_status_agent2	string	attribute
duration_seconds	double	measure
total_steps	bigint	measure
messages_sent	bigint	measure
messages_agent1	bigint	measure
messages_agent2	bigint	measure
total_cost	double	measure
row_hash	string	technical
load_timestamp	timestamp	technical

gold.agg_arena_latest aggregate

The latest rating of a model in a category, with a flag for the newest board.

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
category_sk	bigint	PK, FK to dim_arena_category.category_sk

COLUMN	TYPE	ROLE
date_key	int	measure
rating	double	measure
rating_ci_lower	double	measure
rating_ci_upper	double	measure
vote_count	bigint	measure
rating_rank	int	measure
rating_percentile	double	measure
is_on_latest_board	boolean	flag
load_timestamp	timestamp	technical

gold.agg_style_sensitivity aggregate

A model's rank under the default style against style control, per category.

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
category	string	PK
rating_default	double	measure
rating_style_control	double	measure
rank_default	int	measure
rank_style_control	int	measure
rank_shift	int	measure
load_timestamp	timestamp	technical

gold.agg_frontier_timeline aggregate

The leader and the best rating so far, per category and date.

COLUMN	TYPE	ROLE
category_sk	bigint	PK, FK to dim_arena_category.category_sk
date_key	int	PK, FK to dim_date.date_key
top_rating	double	measure
top_model_sk	bigint	FK to dim_model.model_sk
models_ranked	bigint	measure
best_rating_so_far	double	measure
is_new_leader	boolean	flag
load_timestamp	timestamp	technical

gold.agg_capability_cost aggregate

Latest rating, base-model list price and estimated monthly cost, per scenario.

COLUMN	TYPE	ROLE
model_sk	bigint	PK, FK to dim_model.model_sk
scenario_sk	int	PK, FK to dim_workload_scenario.scenario_sk
arena_rating	double	measure
arena_rating_style_control	double	measure

COLUMN	TYPE	ROLE
benchlm_display_score	double	measure
input_price_per_mtok	double	measure
output_price_per_mtok	double	measure
est_monthly_cost_usd	double	measure
rating_points_per_dollar	double	measure
load_timestamp	timestamp	technical

gold.agg_coordination_by_setting aggregate

Pass rate, duration and merge conflicts per study setting.

COLUMN	TYPE	ROLE
setting_sk	int	PK, FK to dim_study_setting.setting_sk
tasks	bigint	measure
evaluated	bigint	measure
passed	bigint	measure
pass_rate	double	measure
avg_duration_seconds	double	measure
merge_conflicts	bigint	measure
merge_conflict_rate	double	measure
evaluation_coverage	double	measure
load_timestamp	timestamp	technical

gold.agg_coordination_paired aggregate

One task and cooperative setting, compared with the single-agent result of the same task.

COLUMN	TYPE	ROLE
task_sk	bigint	PK, FK to dim_task.task_sk
setting_sk	int	PK, FK to dim_study_setting.setting_sk
passed_setting	boolean	flag
passed_single_agent	boolean	flag
outcome_vs_single_agent	string	attribute
load_timestamp	timestamp	technical

gold.model_alias aggregate

How each Arena model key was mapped to a base model.

COLUMN	TYPE	ROLE
source	string	PK
source_key	string	PK
base_key	string	attribute
method	string	attribute
load_timestamp	timestamp	technical

APPENDIX B

Bronze and Silver summary

The full column lists of both layers are in the `README.md` of the repository. This summary gives the tables, their grain and their keys.

BRONZE TABLE	ONE ROW IS	KEY
`bronze.arena_leaderboard_raw`	A row of an Arena Parquet file	`record_hash`
`bronze.cooperbench_member_raw`	A JSON file from a CooperBench archive	`record_hash`
`bronze.benchlm_raw`	A record of a BenchLM export	`record_hash`
`bronze.epoch_models_raw`	A row of an Epoch CSV snapshot	`record_hash`

SILVER TABLE	PRIMARY KEY	SET ASIDE
`silver.arena_leaderboard`	`subset, category, model_name, leaderboard_publish_date`	224
`silver.epoch_models`	`model, publication_date`	12
`silver.benchlm_models`	`slug`	0
`silver.benchlm_benchmarks`	`benchmark_key, category`	0
`silver.benchlm_pricing`	`canonical_model_key`	0
`silver.cooperbench_members`	`study_setting, member_path`	1
`silver.cooperbench_evals`	`study_setting, member_path`	0
`silver.cooperbench_runs`	`study_setting, member_path`	0

APPENDIX C

Repository map

PATH	CONTENTS
`.github/workflows/frontier-collect.yml`	GitHub Actions: smoke, full and incremental collection
`ingestion/frontier_collect.py`	The collector
`notebooks/00_setup_databricks.ipynb`	One-time setup: five schemas, the volume and its folders
`spark/bronze/frontier_bronze_load.ipynb`	Raw files to Bronze
`spark/silver/frontier_silver_load.ipynb`	Bronze to Silver
`spark/gold/frontier_gold_load.ipynb`	Silver to Gold, optimization evidence and PII scan
`dashboard/`	The Power BI file
`docs/`	Run guides and the Gold design notes
`proof/`	Executed notebooks with saved outputs
`sample_data/`	Small samples with a manifest and source notices
`proposals/`	Proposal documents
`README.md, LICENSE`	Overview, data models, run guide and the MIT licence